TAISAP reframing pass — unified diff

paper/main_taisap.tex and paper/references_taisap.bib
baseline commit 1bee19e → working tree (commit e81e062 and after)

Note: this is a plain unified git diff, not a rendered latexdiff (see qa_report.md for why). Lines beginning - were removed, + were added; unmarked lines are context.

```
diff --git a/paper/main_taisap.tex b/paper/main_taisap.tex
index 5b56a79..136e868 100644
--- a/paper/main_taisap.tex
+++ b/paper/main_taisap.tex
@@ -71,7 +71,7 @@ implementation patterns that satisfy them, report that the strongest defense aga
 usage-authority manipulation costs under 0.1\% of end-to-end time with real inference,
 and delimit the formal model, extended with a second, causal-ordering specification of
 the asynchronous settlement mechanism. The contribution is a systematic security
-analysis of deployed AI-serving infrastructure, not a new model-level attack: B0 is a
+analysis of deployed AI-serving infrastructure: B0 is a
 known baseline, and M1/M2 systematize, formalize, and measure mechanisms with prior
 individual precedent.
 \end{abstract}
@@ -113,28 +113,33 @@ accounting, asynchronous settlement, formal verification, TLA+, cloud billing in

 \section{Introduction}
 \label{sec:intro}
-Metered computational services sell work whose quantity is not known when the work is
-authorized. A client asks for computation, the service performs and delivers it, and only
-afterwards does the system learn what to charge. Between those two moments the service
-holds economic state that is authoritative for billing but not yet durable, and that
-state is read, written, and reconciled by several components: an authorization path, an
-execution path that delivers value incrementally, and a settlement path that may run
-asynchronously behind both. Whether the service can be relied on to charge for what it
-delivered is therefore a question about distributed state management, not about
-accounting policy.
-
-LLM inference serving makes the question concrete and current. The billed unit, the
+LLM inference APIs bill for tokens that are produced at runtime and streamed to the
+client before the charge is settled, which places the metering and billing path inside
+the attack surface of the AI-serving system. The billed unit, the
 token, is produced at runtime by an opaque, streamed, long-running computation delivered
 to a remote client over a network; the quantity is a multi-category vector; and the
 usage record that determines the charge is itself produced by the serving system while
 the response is still in flight -- so the authorization, execution, and settlement paths
-above now carry security-critical state, making the usage-metering and settlement
+carry security-critical state, making the usage-metering and settlement
 subsystem part of the AI-serving system's security boundary rather than a peripheral
 accounting concern external to it. Mistakes in how that state is
 partitioned across components can produce integrity failures even when every individual
 component computes usage correctly, which is why this paper studies the accounting
 lifecycle itself: its states, its transitions, the conditions under which it is safe, and
 what happens empirically when those conditions are broken.
+
+This is a structural property of metered computational services generally, not an
+accident of one implementation, which is why the lifecycle we study is stated over the
+request lifecycle of a metered service rather than tied to language-model inference.
+Such services sell work whose quantity is not known when the work is
+authorized. A client asks for computation, the service performs and delivers it, and only
+afterwards does the system learn what to charge. Between those two moments the service
+holds economic state that is authoritative for billing but not yet durable, and that
+state is read, written, and reconciled by several components: an authorization path, an
+execution path that delivers value incrementally, and a settlement path that may run
+asynchronously behind both. Whether the service can be relied on to charge for what it
```

```
+delivered is therefore a question about distributed state management that the LLM case
+above makes concrete and security-relevant, not merely an accounting-policy question.
 Existing security
 work on this surface runs in three directions, all of which assume the client is honest,
 impersonated, or irrelevant. Provider-side auditing asks whether a dishonest provider can
@@ -172,15 +177,16 @@ the accounting lifecycle, not the model.
 \textit{Contributions.}
 \begin{enumerate}
 \item An accounting-state model of a metered service request
-(Section~\ref{sec:statemodel}) connecting execution, value delivery, authorization,
+(Section~\ref{sec:statemodel}), motivated by token-metered, streamed LLM inference,
+connecting execution, value delivery, authorization,
 reservation, debit, reconciliation, and refund, with a single integrity property,
 $V(r) \le N(r)$, that admits legitimate refunds. The model is stated over the request
 lifecycle of a metered service and is not specific to language-model inference.
 \item A taxonomy of three experimentally distinguishable failure dimensions
 (Section~\ref{sec:taxonomy}) -- synchronization, commitment timing, and usage authority --
 each localized to a distinct lifecycle element (a shared-state transition, a terminal
-path, a data-authority boundary), with known, systematized, and defensive boundaries
-stated explicitly.
+path, a data-authority boundary) of a token-metered LLM-serving request, with known,
+systematized, and defensive boundaries stated explicitly.
 \item Machine-checked verification of the lifecycle (Section~\ref{sec:formal}) in TLA+,
 checked exhaustively with TLC over ten configurations and four invariants, producing
 counterexamples that match the measured failures and an exhaustive orthogonality result
@@ -298,7 +304,10 @@ Pay-as-you-go pricing is a defining feature of cloud services \citep{armbrust},
 by deciding whether to admit work, observing usage, and committing a charge -- a
 placement of responsibilities that determines security \citep{bass}; the failures studied
 here follow from where that placement sits in the request path, not from a defect in any
-single component. Existing work covers the other three quadrants of
+single component. The direction of error we study is under-payment by the client; the
+opposite direction, a platform over-accounting resource usage against its own declared
+billing model, is also documented and is unrelated to any client action
+\citep{serverlesscosts}. Existing work covers the other three quadrants of
 Table~\ref{tab:quadrant}. CoIn \citep{coin} shows reasoning tokens invisible to the client
 can be counted and audited only with effort; \citet{invisible} argues opaque LLM services
 need audit mechanisms because the billed quantity is unverifiable to the payer; and
@@ -422,6 +431,34 @@ an honest-provider threat model across representative metering architectures, wi
 common measurement and an ablation of the enforcement primitives. This is a bounded
 negative from a targeted search. It does not establish that no such work exists.

+\subsection{Scope and novelty}
+This paper is not a model-level attack: it does not target the weights, outputs, or
+training data of the language model, and no manipulation in our harness depends on
+prompting, jailbreaking, or extracting information from the model itself. The object
+under study is the serving-layer accounting boundary -- the authorization, execution,
+and settlement paths that sit between the model and the client's bill -- which is a
+property of the AI-serving infrastructure, not the model. We take this to be an
+AI-security problem rather than a general distributed-systems one for three reasons
+specific to LLM serving: the billed quantity is produced at runtime by the model's own
+execution rather than known in advance, so the accounting boundary cannot be separated
+from inference itself; the usage vector is multi-category and provider-defined (input,
+cached input, output, and in some systems reasoning tokens), a structure particular to
+how LLM providers price inference rather than a generic metering scheme; and the
+quantity is disputable because tokenization is nondeterministic across engines and
+providers \citep{petrov}, which has no direct analogue in metered services whose billed
+unit is a fixed physical quantity such as compute-seconds or bytes. Independent evidence
+supports treating the serving layer itself as attack surface: internet-wide measurement
+of deployed LLM endpoints finds authentication and encryption frequently missing at the
+serving layer \citep{unveilinglandscape}, and serving systems that share concurrency,
+caches, and scheduling state across tenants have an independently documented,
+security-relevant attack surface \citep{grief,gift} -- the same shared runtime state this
+paper's accounting lifecycle reasons about, applied here to the economic rather than the
+confidentiality or availability dimension. B0 is a known baseline mechanism, included for
```

```
+comparison rather than as a contribution; M1 and M2 are not new attack primitives either,
+but their systematization, exhaustive formal verification, and measurement under the
+LLM-serving usage model specifically -- runtime-produced, multi-category,
+tokenizer-disputed -- has not, to our knowledge, been done before.
+
 \section{Accounting State Model}
 \label{sec:statemodel}
 We model a metered inference request as a state machine (Figure~\ref{fig:statemachine})
@@ -1537,7 +1574,7 @@ reserve-reconcile path are not characterized by this same cost measurement.
 \section*{Acknowledgements}
 No external acknowledgements apply to this work.

-\section*{CRediT authorship contribution statement}
+\section*{Author Contributions}
 \textbf{Hardik}: Conceptualization, Methodology, Software, Investigation, Formal
 analysis, Data curation, Validation, Visualization, Writing -- original draft, Writing --
 review and editing. \textbf{Natasha Kaila}: Investigation, Writing -- review and editing.
diff --git a/paper/references_taisap.bib b/paper/references_taisap.bib
index 0775346..0ca415f 100644
--- a/paper/references_taisap.bib
+++ b/paper/references_taisap.bib
@@ -568,3 +568,31 @@
   title = {Experimentation in Software Engineering. Springer, Berlin, Heidelberg.},
   howpublished = {},
 }
+
+@misc{unveilinglandscape,
+ author = {Hou, X. and Han, J. and Zhao, Y. and Wang, H.},
+ year = {2025},
+ title = {Unveiling the landscape of LLM deployment in the wild: an empirical study. arXiv:2505.02502.},
+ howpublished = {},
+}
+
+@misc{grief,
+ author = {Zhao, Y. and Zhao, Y. and Zhang, Y. and Liu, Z. and Mazurek, M.L.},
+ year = {2026},
+ title = {Continuous discovery of vulnerabilities in LLM serving systems with fuzzing. arXiv:2605.11202.},
+ howpublished = {},
+}
+
+@misc{gift,
+ author = {Shi, J. and Wang, X. and Tan, C. and Gu, J.},
+ year = {2026},
+ title = {Here is a GIFT: enforcing user data isolation in LLM serving via GPU information flow tracking. arXiv:2608.25431.},
+ howpublished = {},
+}
+
+@misc{serverlesscosts,
+ author = {Lin, C. and Ma, Y. and Shahrad, M.},
+ year = {2025},
+ title = {Demystifying serverless costs on public platforms: bridging billing, architecture, and OS scheduling. arXiv
+     :2506.01283.},
+ howpublished = {},
+}
```